

Actividad 3.2. Analizador sintáctico, semántico y generación de código intermedio

Propósito

Completar la implementación de un compilador de un lenguaje de programación representativo hasta la generación de código intermedio.

Instrucciones

1. Completar la implementación de un analizador sintáctico para el subconjunto del lenguaje C que se utilizó en clase y algunas extensiones al mismo. Considera el siguiente documento que describe la gramática de base y su extensión provisto en el archivo PDF adjunto.
2. Implementar un generador de código intermedio usando la biblioteca LLVM Lite. Considera también el soporte al llamado de la función "printf", como lo ilustra el siguiente gist:
<https://gist.github.com/alendit/defe3d518cd8f3f3e28cb46708d4c9d6>
3. Cubrir un construcciones básicas del lenguaje: estructuras de control (IF/ELSE, WHILE, FOR, DO/WHILE y SWITCH), definición y llamado a funciones, y conversión implícita de tipos.
4. Como prueba de concepto, escribe 5 programas que demuestren la capacidad de tu compilador para soportar programas con llamadas recursivas, p.e., factorial, serie de Taylor, serie de Fibonacci, etc.
5. Escribe un reporte del proyecto, describiendo cada una de las etapas y el código clave en cada una de ellas. Incluye un reflexión personal de lo aprendido en el proyecto.

Especificaciones

- **Duración estimada:** 8 horas.
- **Modalidad:** Individual
- **Formato de entrega:** Un archivo ZIP con el código fuente de los analizadores léxico, sintáctico y generador de código intermedio, y un reporte que explique el código, cómo ejecutarlo, una reflexión personal y la liga al repositorio Git en el que matuvieron su código. En el Git, se debe mostrar que participó de manera equitativa en el desarrollo de la solución. La falta participación o participación no equitativa implica una calificación

reprobatoria. Incluye un código que usaste para probar tu compilador (mínimo de 5 programas de ejemplo).

- **Medio de entrega:** Se entrega en Canvas en el botón "Entregar Tarea" en la parte superior de esta pantalla.

Evaluación

Tu código fuente debe compilar sin errores ni warnings y debe ejecutarse correctamente. Piensa en posibles casos de prueba extremos que pueden ser utilizados para probar tu programa. Para realizar tus pruebas, utiliza los archivos que encontrarás aquí.

Tu solución propuesta debe ser correcta y eficiente. En el repositorio Git, debe reflejarse el proceso de desarrollo. Como partes del proyecto son desarrolladas en equipo, es necesario que el historial de commits refleje tu participación equitativa. Una participación no equitativa será penalizada con una calificación reprobatoria. Como sugerencia, puedes considerar hacer un commit después de avanzar una regla de producción en las distintas etapas, como lo hicimos en clase.

Para obtener el 100% de los puntos de esta actividad, tu programa:

- 80% - Tu código obtendrá:
 - Excelente (80%) - Si evalúa correctamente los 4 casos de prueba.
 - Muy Bien (60%) - Si evalúa correctamente 3 casos de prueba.
 - Bien (40%) - Si evalúa correctamente 2 casos de prueba.
 - Insuficiente (20%) - Si evalúa correctamente 1 o 0 casos de prueba.
- 10% - El código deberá seguir los lineamientos estipulados en el estándar de codificación [Descargar estándar de codificación](#).
- 10% - Manual de usuario de tu solución.

Fecha de entrega: 26 de mayo hasta las 23:59 h (Hora de la CDMX)

Medio: Correo electrónico (luciano.garcia@tec.mx)